FIAP ON

DATA INGESTION

# INTRODUÇÃO A INGESTÃO DE DADOS

12

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

## LISTA DE FIGURAS

Figura 1 - Lifecycle of data engineering ..... 9
Figura 2 - Conceitos sobre data engineering e data Science ..... 10
Figura 3 - IOT Devices e fila de Figura 3: IOT Devices mensagens..... 13
Figura 4 -Tipos de dados analíticos ..... 15
Figura 5 - Pipeline de dados..... 16
Figura 6 -Pipeline de dados com Airflow, Spark e Hadoop ..... 17
Figura 7 - Ingestão de dados do ambiente transacional para o datalake ..... 22
Figura 8 - Tela de gerenciamento do Airflow..... 25

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

## LISTA DE CÓDIGOS FONTE

Código-fonte 1 - Comando para baixar imagem airflow ..... 18
Código-fonte 2 - Código do arquivo de configuração do Airflow..... 20
Código-fonte 3 - Docker ps lista os contêineres em execução..... 20
Código-fonte 4 - Docker cp para copiar o arquivo da dag para o container do airflow..... 20
Código-fonte 5 - Código de criação da DAG ..... 25

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

**SUMÁRIO**

**1 POR QUE FAZER INGESTÃO DE DADOS ..... 5**
**1.1 Contexto ..... 6**
**2 CONCEITO DE INGESTÃO DE DADOS ..... 9**
**3 TIPOS DE INGESTÃO DE DADOS ..... 11**
**3.1 Carga Full ou Incremental ..... 12**
**4 COMPONENTES DA INGESTÃO DE DADOS ..... 14**
**4.1 Gerenciamento de dados e metadados ..... 15**
**4.2 Data Quality ..... 16**
**4.3 Escalabilidade na ingestão de dados ..... 17**
**5 CRIANDO UMA INGESTÃO DE DADOS FULL LOAD ..... 18**
**6 INGESTÃO DE DADOS ..... 22**
Referências ..... 27

**PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875**
**marcosdaguealmeida@outlook.com**

# TIPOS, CARACTERISTICAS E COMPONENTES DA INGESTÃO DE DADOS

## 1 POR QUE FAZER INGESTÃO DE DADOS

Em nosso cenário de tecnologia atual, é natural a movimentação de dados entre ambientes. Justamente pelo propósito dos ambientes serem diferentes, há a necessidade dessa movimentação para atender às diferentes demandas de negócios.

Se pensarmos em um ambiente transacional, temos os dados de transações recentes, esses dados atendem à demanda de fazer a roda girar, de fazer a empresa funcionar, então, são dados de cadastro de clientes, produtos, vendas, cancelamentos e inúmeros outros dados que atendem a operação da companhia em seu dia a dia.

Não é difícil imaginar porque esse ambiente não é adequado para grande parte dos projetos de dados. Imagine você criando um pipeline de dados que faz a leitura de algumas dezenas de tabelas, processa isso para alimentar um modelo de machine learning. Nada legal, né?

Por isso esse dado precisa ser movimentado para um ambiente adequado onde possamos manipula-los de forma que ele se torne adequado à necessidade do nosso projeto e sem causar prejuízo ao ambiente transacional.

Essa movimentação de dados passou pela criação de novos conceitos por ter necessidades especificas em cada projeto. Temos o nosso famoso ETL (Extract, Transform and Load), onde o foco está em movimentar os dados do ambiente transacional para o ambiente de Business Intelligence já processado e transformado, temos o ELT (Extract, Load and Transform), para casos onde há a necessidade de ter o dado RAW, ou seja, sem transformação também no mesmo ambiente, mas ainda temos uma etapa de transformação obrigatória nesse processo, já a ingestão de dados geralmente tem como destino um Datalake, e esse processo movimenta os dados sem nenhum tipo de transformação nos dados para que esse dado seja centralizado em um ambiente e possa ser usado em diferentes projetos, já que estará em seu formato natural.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

## 1.1 Contexto

Um dos principais desafios da área da computação é armazenar as informações de forma eficiente, segura e de fácil acesso. Na busca desse objetivo, passamos por bancos de dados otimizados de diversas formas, desde sofisticados mecanismos de busca à persistência de informações, novos paradigmas foram criados para forma como os dados são armazenados, em formato colunar, tabular, Json, chave-valor, armazenados em arquivos de dados em formato Parquet, ORC, datafiles separados logicamente, fisicamente e por aí vai.

Com esse contexto em mãos, podemos ver que temos dados de diversos formatos, vindos de diversas origens, que exigem tratamentos distintos, junções, agregações, filtros e tratativas para se chegar ao produto de dados final, que muitas vezes é uma tabela que será consumida por um outro processo.

Quando observamos o que é ingestão de dados, parece simples, mas nem sempre é. A tomada de decisão que se baseará nesses dados, precisa que esse dado tenha sido manipulado corretamente. Imagine você levar o dobro de registros a mais em uma carga, ajustar o data type de colunas e levar um valor diferente do correto; isso pode levar a uma tomada de decisão errada.

Imagine o cenário onde temos um modelo de machine learning que gera uma probabilidade de cancelamento de contrato por parte dos clientes de uma empresa de Streaming e algumas das variáveis mais importantes desse modelo são o tempo de utilização da plataforma e títulos assistidos. Esses dados são observados pelo modelo em uma linha temporal dos últimos 6 meses; a ingestão de dados para esse ambiente sofreu uma quebra e carregou dados dos últimos dois meses de forma errada, apresentando uma queda no uso da plataforma e títulos assistidos pelos clientes. O resultado disso? Muitos clientes com alta probabilidade de cancelamento de acordo com o modelo de predição de churn. Esse tipo de problema infelizmente não é tão incomum quanto parece, mas o que temos que ter em mente sempre é que os dados são a matéria-prima de um cenário data driven, logo, se o dado não tiver qualidade, os insumos gerados também não terão.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Dito isso, seguem algumas responsabilidades necessárias para fazer uma ingestão de dados com qualidade:

- **Entendimento do escopo da ingestão de dados**: parece óbvio carregar dados de um lado para outro, mas com que frequência? Qual impacto se essa carga falhar? Quem vai consumir esses dados? Por qual ferramenta? De qual forma? Essas são algumas perguntas a serem respondidas para ter uma ingestão de sucesso. Lembre-se que quem vai consumi-los pode ser uma área de negócios, então, entender as necessidades, nuances e requisitos é de extrema importância;
- **Controle de custos**: carga full, cargas incrementais, real time e batch são algumas opções que temos e que podem mudar drasticamente nossos custos com ingestões de dados. Fique atento para conhecer bem seu projeto e dimensionar os custos de acordo com suas necessidades;
- **Conhecimento em devops/dataops**: ter conhecimento em metodologias e abordagens mais atuais é de suma importância. Alguns exemplos de técnicas são Devops, Dataops, bem como CI e CD (Continuous Integration e Continuous Delivery).

## 1.2 Evolução da ingestão e engenharia de dados

O ETL (Extract, Transformation and Load) surgiu na década de 70 quando se obteve a necessidade de centralizar os dados em um único local. Antes disso, os dados ficavam em bancos de dados separados e sem um processo de organização, agregação e geração de valor como nos métodos atuais.

Com sua popularidade, o ETL se tornou o método padrão para coletar dados de diferentes origens e transformá-los antes de carregar em um ambiente centralizado. Com o passar dos anos, foram surgindo novos dados, formatos, ferramentas e necessidades especificas, e hoje, o ETL é mais uma forma de fazer ingestão de dados, mas não a única.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Nesses últimos anos, passamos por várias fases envolvendo a adoção de dados na tomada de decisão: o BI, com dados processados em data warehouses e cubos analíticos trazendo uma visão do passado sobre um problema, depois chegamos à era de Big Data, onde centralizar tudo - era o que tinha que ser feito -; esse foi o ponto alto da ingestão de dados.

Empresas construíram data lakes gigantescos com petabytes de dados, e que a maior parte deles não era tão utilizado assim, e chegamos à era do data centric, onde o foco é o objetivo dessa abordagem: focar naquilo que vamos usar e de uma forma mais sustentável com maior governança, processos, automação e qualidade.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

## 2 CONCEITO DE INGESTÃO DE DADOS

Para contextualizar melhor nosso cenário de ingestão de dados, vamos partir de uma imagem contendo o ciclo de vida da engenharia de dados, área responsável pela ingestão de dados.

Na imagem abaixo, conseguimos entender o ciclo de vida do processo de ingestão de dados, com o começo, meio e fim.

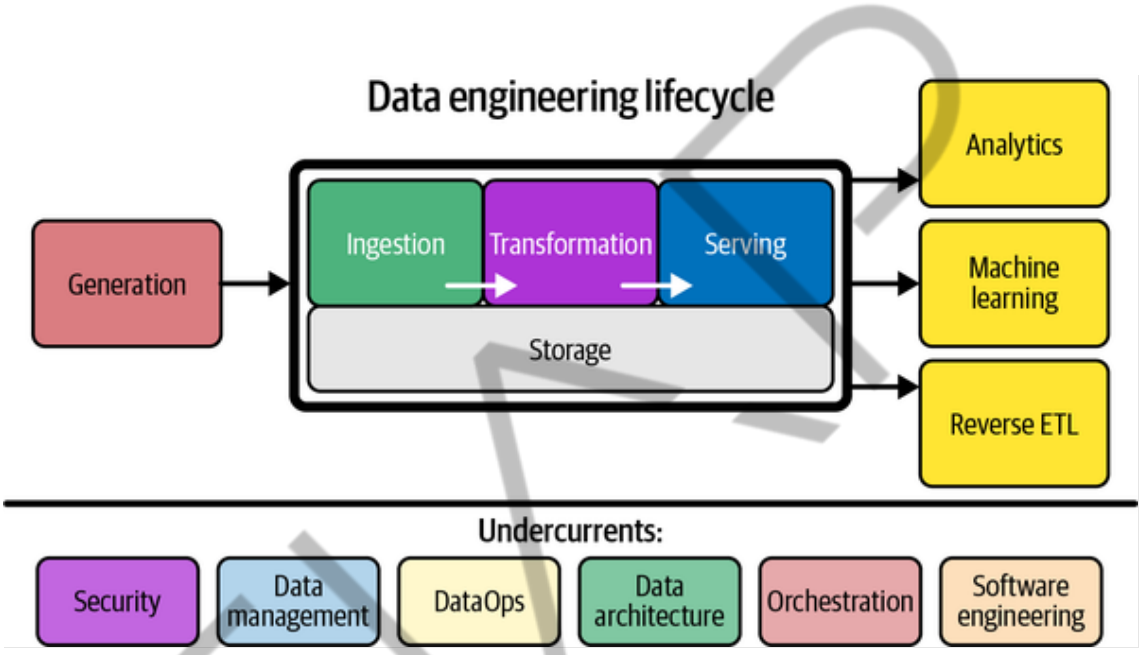


Figura 1 - Lifecycle of data engineering
Fonte: Fundamentals of Data Engineering, 2022

Esse ciclo de vida, de acordo com Reis, Joe e Housley e Matt, é composto basicamente por:

- Generation;
- Storage;
- Ingestion;
- Transformation;
- Serving.

Mas e quanto às etapas de machine learning e analytics? Fazem parte do ciclo de vida? Sim, mas não diretamente. A engenharia de dados é uma subdisciplina da ciência de dados, estando conectada com alguma dependência entre elas, uma vez que na engenharia de dados disponibilizamos os dados para

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

que sejam consumidos e transformados em produto de dados final pela ciência de dados.

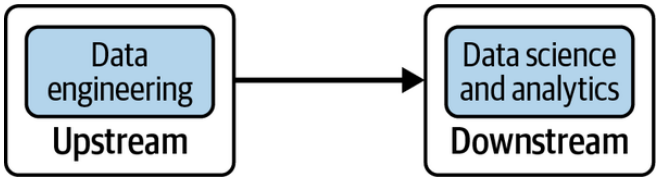


Figura 2 - Conceitos sobre data engineering e data Science
Fonte: Fundamentals of Data Engineering, 2022

Impossível falar sobre ingestão de dados e engenharia de dados e não falar sobre alguns outros temas que tangem essa tarefa de ingestão de dados, como:

- Segurança da informação;
- Gerenciamento de dados mestres;
- DataOps;
- Arquitetura de dados;
- Orquestração de pipelines de dados;
- Engenharia de software.

Muitos devem estar se perguntando o porquê de todo esses itens. Você, como responsável por transitar esses dados entre ambientes, deve ser responsável por pensar em formas seguras de fazer isso, bem como gerenciar dados mestres, dados dos quais a empresa não sobreviveria caso fossem perdidos, assim como uma estrutura robusta de pipelines, seguindo as boas práticas de os operacionalizar. Uma arquitetura de dados proporciona maior organização e escalabilidade no que diz respeito aos dados, por isso, antes de movimenta-los, crie uma arquitetura que atenda esse cenário. Orquestrar os pipelines de dados que por vezes deverão vir de diversas origens diferentes e devem necessitar de tratamentos específicos explica porque orquestrar esses passos necessários são de suma importância. E por fim, muitas vezes, tudo isso que falamos até o momento precisa de algo que una todas as coisas. Em muitos cenários, criaremos uma estrutura robusta de back-end para garantir a integridade de toda essa operação.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

# 3 TIPOS DE INGESTÃO DE DADOS

Para entendermos os tipos de ingestão de dados, precisamos primeiramente entender suas fontes e destinos, pois, a partir disso, podemos discutir necessidades, estratégia e tomar decisões sobre qual melhor forma de fazê-lo e, inclusive, pensar em ferramentas para atender a isso.

Vamos imaginar o primeiro cenário, uma grande fábrica com diversas máquinas equipadas com sensores que enviam dados para um hub de dados, essas máquinas devem ser monitoradas e a cada falha apresentada algum técnico deve ir até ela imediatamente para corrigir o erro.

Agora vamos imaginar um segundo cenário, onde temos dados de vendas provenientes do CRM. Logo, temos dados de vendas, Leads, clientes em fase de negociação e temos a quantidade de clientes que cancelaram seus produtos. A alta gestão da empresa precisa de um relatório contendo uma visão de CAC (Custo de aquisição de cliente) versus Churn (Cancelamento de clientes) por mês, dos últimos 6 meses.

Espero que esteja claro que os dois projetos têm necessidades especificas: em um precisamos do dado o mais rápido possível para corrigir uma falha em uma máquina, no outro precisamos de um relatório mensal de um período para a tomada de decisão. Dessa forma, um dado deve ser carregado em tempo real, ou seja, à medida que os eventos ocorrem, os dados devem ser carregados no nosso banco de dados e ser disparado um e-mail, um alerta de alguma forma para que seja providenciado o reparo do equipamento; o outro, podemos carregar e processar os dados uma única vez ao mês.

Isso chamamos de cargas em real-time, ou seja, tempo real e batch ou lote. As cargas em real time podem sofrer um gatilho de dados, que é enviado imediatamente ao banco de dados quando um novo dado é salvo ou pode ser de tempos em tempos, como a cada 30 segundos, 60 segundos, o que a torna, de certa forma, em uma carga batch, que é executada a cada X tempo, por exemplo, todo final de semana, os dados são carregados e processados para gerar o relatório. Ou a cada dia, a cada hora; isso é uma carga em batch.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

### 3.1 Carga Full ou Incremental

Agora que definimos periodicidade de carga, se será sempre que um novo evento ocorrer ou em quantidades geradas de tempos em tempos, precisamos definir o tipo de carga do ponto de vista dos dados. Levaremos todos os dados sempre ou levaremos apenas o que é um dado novo? Chamadas cargas full ou cargas incrementais, também fazem parte da estratégia, se seu dado do passado é imutável, ou seja, se os dados que foram gerados semana passada não passaram por atualização, não há a necessidade de carregar todos os dados, todas as cargas, carregue apenas os dados novos. Agora, se seu dado pode passar por uma atualização, por exemplo, nos dados do CRM, o cliente pode sair do estágio de “em negociação” para “realizado”, e a empresa queira contabilizar os dados sempre pela última visão, talvez valha a pena pegar todos os dados em todas as cargas para ter sempre a última versão do dado.

Nesses cenários, não há receitas pré-montadas para atender a tudo e todos, cada projeto tem suas necessidades e, a partir delas, criamos uma estratégia para atendê-lo.

Um cenário onde é preciso bastante cautela no uso de cargas de dados em real time é quando falamos de IOT ou envio de geolocalização em tempo real. Imagine você trabalhando em uma empresa que presta serviços de entregas com motoboys freelance e cada entrega é monitorada enviando a geolocalização a cada 5 segundos. A empresa conta com uma frota de 10 mil motoboys parceiros.

Imagine receber 10 mil coordenadas a cada 5 segundos para salvar no banco de dados e apresentar para seu cliente na tela do aplicativo dele.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Para fazer isso, é necessário ter estratégia, necessário criar uma fila e planos de execução dessa carga no banco de dados para evitar que se gere gargalos nele.

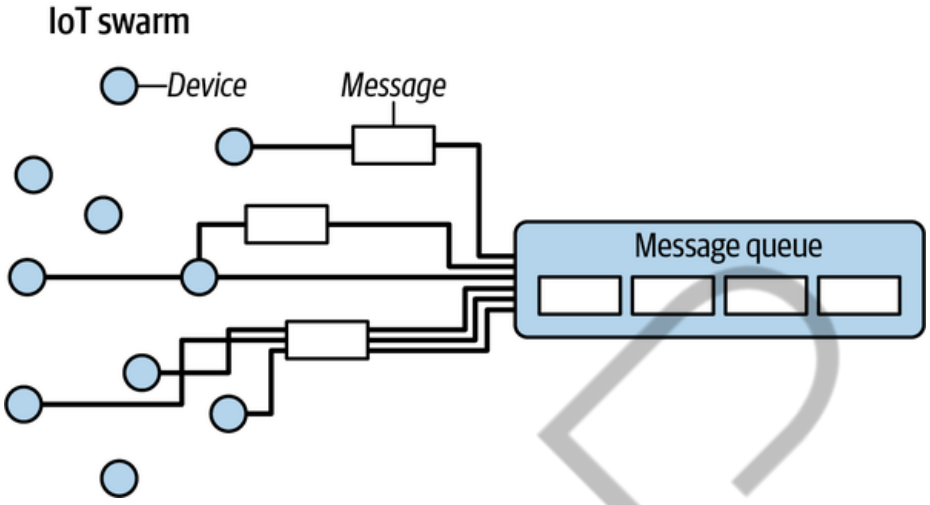


Figura 3 - IOT Devices e fila de Figura 3: IOT Devices mensagens
Fonte: Fundamentals of Data Engineering, 2022

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

# 4 COMPONENTES DA INGESTÃO DE DADOS

Agora que conhecemos os conceitos e tipos de ingestão de dados, vamos falar sobre componentes desse tipo de tarefa.

O primeiro componente, apesar de não ser um componente técnico e já ter sido citado anteriormente, vale a pena recapitular: é o entendimento do negócio, do porquê essa ingestão deve existir, a que proposito ela atende, quem vai consumir, como vai consumir, qual impacto dessa carga no negócio. Essas questões nos trazem clareza do porquê estamos fazendo aquela ingestão de dados, os benefícios, impactos e para que serão usadas, isso nos ajuda a direcionar ferramentas e técnicas adequadas nesse processo.

Em armazenamento de dados, assim como falamos das nossas cargas de dados, temos necessidades especificas e que, para atender 100% da expectativa do projeto, é necessário pensar nisso também. A leitura de dados no disco é sempre um gargalo nos bancos de dados, logo, se seu projeto precisa de dados real time e de rápido acesso, não negligencie o armazenamento dos dados, pois nada vai adiantar ter um processo que rode o pipeline em real time se seu armazenamento não vai ajudar a entregar o que precisa.

Transformações nos dados são necessárias? E se forem, devem ocorrer em que momento? Se seus dados forem passar por algum tipo de tratamento, isso deve contemplar diversos fatores de falhas, planos de rollback para voltar em caso de encontrar algo que comprometa a integridade dos dados.

Data quality, garantir que os dados cheguem ao outro lado da forma esperada, é muito importante. Muitas ferramentas de mercado possuem essa funcionalidade nativa para garantir ao menos de forma quantitativa uma etapa de data quality. Mas nada impede de evoluir esse processo por meio de um processo qualitativo, por exemplo, gerar estatísticas, calcular um IQR sobre algumas colunas importantes dos dados movimentados caso ultrapasse os limites inferior ou superior calculados, gerar um alerta.

Cuidado ao fazer ingestão de dados sem saber qual o propósito do dado. Diferentes fins podem exigir diferentes tratativas para os dados, inclusive, em

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

alguns cenários, exigir o dado em seu formato mais bruto e em outros, os dados processados e transformados.

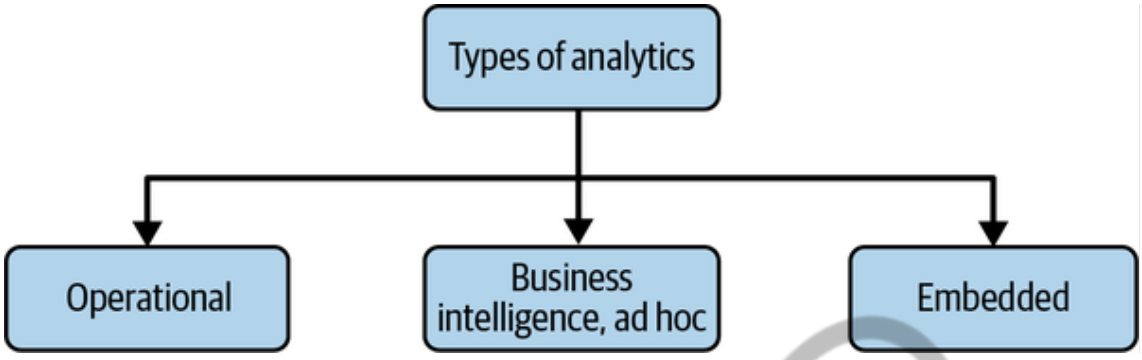


Figura 4 -Tipos de dados analíticos
Fonte: Fundamentals of Data Engineering, 2022

Analytics é core dos produtos de dados. Embora quando falamos de análise de dados de modo geral logo pensamos em BI, atualmente, o cenário não é só esse. Para Joe Reis e Matt Housley, há separações entre análise operacional, business Intelligence, análises ad-hoc e análises incorporadas.

## 4.1 Gerenciamento de dados e metadados

O que antes era uma realidade apenas para grandes empresas, hoje é uma necessidade primordial para empresas de todos os tamanhos. A governança de dados é um assunto que é sensível a muitas organizações, pois demanda tempo, investimento e não surte efeito diretamente no cliente final. Mas ter uma governança de dados, gerenciamento de dados, controle de acesso, gerenciamento de metadados, gerenciamento de qualidade de dados e dados mestre é de suma importância para qualquer empresa, de qualquer porte, e independente do nível de adoção do uso de dados que esteja, quanto antes implantar uma política de governança de dados melhor.

De acordo com o DMBOK do DAMA (Data Management Association International), o gerenciamento de dados é o desenvolvimento, a execução e a supervisão de planos, políticas e práticas que fornecem, controlam, protegem e aumentam o valor dos dados e dos ativos de informação ao longo do seu ciclo de vida.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

## 4.2 Data Quality

Os dados são a matéria-prima para geração de um produto de dados e, como qualquer produto final, se a matéria-prima não for de qualidade, o produto final também não será. Ao movimentar dados de um ambiente para outro, podemos passar por diversos problemas, desde perda de informação e quebra no pipeline de dados a tratativas que estão sendo feitas e tiveram um comportamento fora do esperado. Para lidar com isso, é necessária a criação de mecanismos que detectem essas anomalias e que possam alertar de alguma forma o time responsável para que possa ser regularizado. A maior parte das ferramentas de mercado oferece isso de forma nativa, mas caso haja necessidade, é sempre bom criar algum processo que seja mais específico para você, veja o exemplo abaixo:

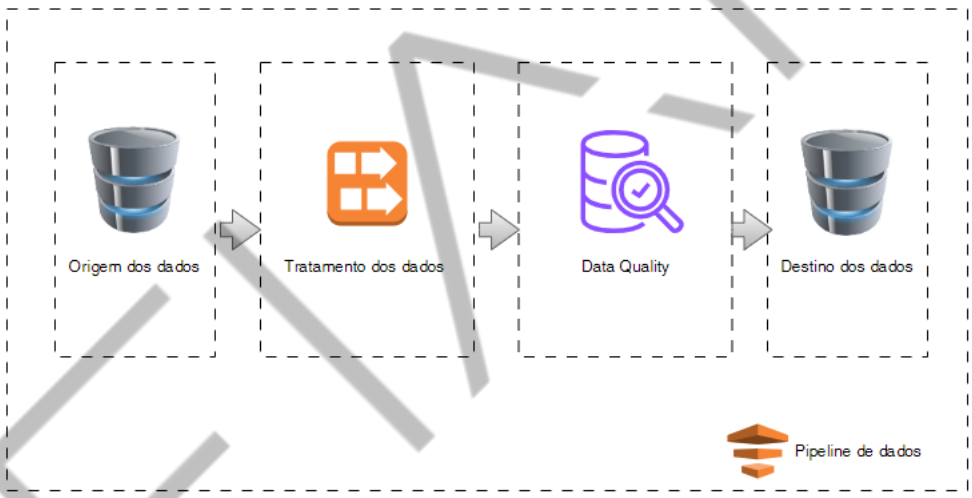


Figura 5 - Pipeline de dados
Fonte: Elaborado pelo autor

Aqui vemos um pipeline de forma macro, onde temos as origens de dados, que podem ser ambientes de bancos de dados transacionais. Os dados são submetidos a uma etapa de tratamento. Por exemplo, para ajustes de data types das colunas das tabelas e outras tratativas, passamos pela etapa de qualidade de dados, onde são checadas quantidades e algumas estatísticas sobre eles. Exemplo: em um pipeline de dados que carrega dados de vendas semanais, geralmente temos uma quantidade de vendas que varia entre 150 e 450. Em uma carga de dados, ocorreu uma quantidade de 1500 vendas em uma semana. Isso quer dizer que as vendas não podem aumentar? Sim, podem, mas

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

geralmente isso não acontece com números tão exponenciais e isso é, para nós, alerta que temos um comportamento diferente do padrão.

### 4.3 Escalabilidade na ingestão de dados

Natural iniciarmos um projeto com uma quantidade de registros sendo processados diariamente e esse número crescer absurdamente. Isso faz parte do nosso dia a dia, logo, é necessário pensar nesse tipo de problema e se preparar para dar escala ao seu projeto quando e como for necessário.

Um exemplo interessante e bastante atual é o uso do Apache Airflow, onde conseguimos montar fluxos de dados que inclusive podem ser executados em containers. O Airflow trabalha com DAGs para execução e cada DAG tem seu papel dentro do pipeline de dados.

Abaixo, temos um exemplo de um pipeline de ingestão de dados usando Airflow na orquestração, Spark no processamento e Hadoop no armazenamento.

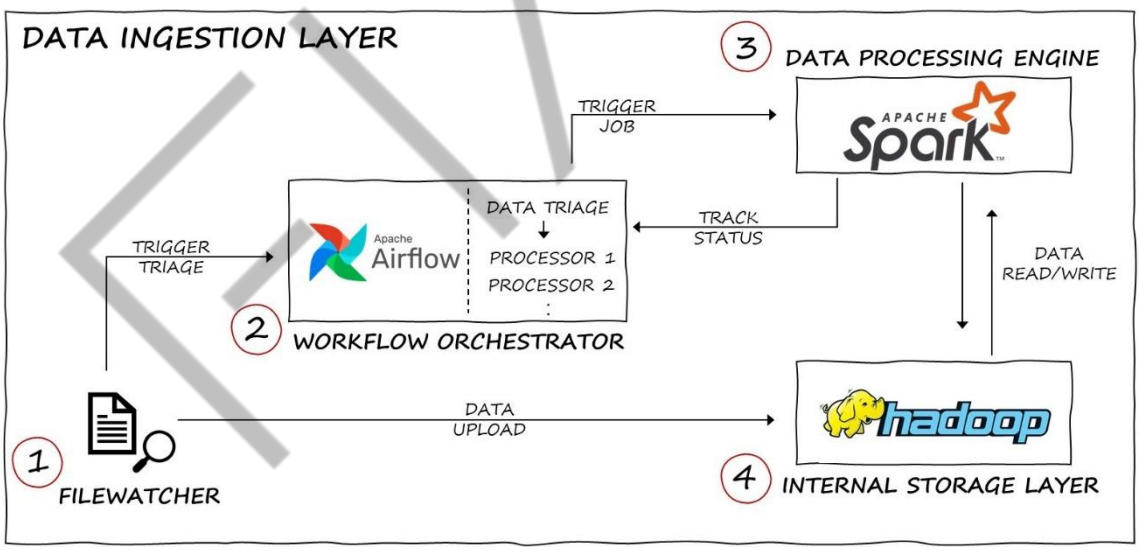


Figura 6 -Pipeline de dados com Airflow, Spark e Hadoop
Fonte: Goals101 - https://www.goals101.ai/2020/06/selecting-the-right-tech-stack-for-a-data-pipeline-solution/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

# 5 CRIANDO UMA INGESTÃO DE DADOS FULL LOAD

Para orquestrar e processar a carga de dados, usaremos um software chamado Airflow juntamente com Docker. Usaremos a imagem padrão do Airflow para Docker, o instalador do Docker pode ser baixado em https://www.docker.com/products/docker-desktop/.

O Airflow é uma ferramenta open source de criação de fluxos de tarefas bastante utilizada na área de dados. A principal característica do Airflow é a capacidade de definir, programar e monitorar fluxos, chamados de DAG (Directed Acyclic Graphs).

Temos três elementos básicos no Airflow:

- Operadores – Executam uma ação ou enviam informação para outro sistema executar uma ação dentro do seu Worlkflow;
- DAG – Grafo acíclico onde todas as tarefas são executadas e organizadas de forma que reflete seus relacionamentos e dependências;
- Tasks – Tarefas dentro de uma DAG que devem executar um comando específico para cada tipo de operador.

Para baixar a imagem padrão do Airflow no Docker, abra o terminal e execute o comando abaixo:

```
docker pull puckel/docker-airflow
```

Código-fonte 1 - Comando para baixar imagem airflow
Fonte: Elaborado pelo autor

O próximo passo é criar um arquivo de configuração chamado DockerComposeCeleryExecutor.yml. Dentro desse arquivo, temos tudo que é necessário para executar o container do Airflow.

Esse arquivo baixa e instancia quatro imagens para subir o serviço do Airflow.

Veja abaixo alguns trechos do arquivo, onde é instanciado os serviços necessários como Redis, Postgres, Scheduler e Flower.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
version: '2.1'
services:
  redis:
    image: 'redis:5.0.5'
    # command: redis-server --requirepass redispass

  postgres:
    image: postgres:9.6
    environment:
      - POSTGRES_USER=airflow
      - POSTGRES_PASSWORD=airflow
      - POSTGRES_DB=airflow
    # Uncomment these lines to persist data on the local filesystem.
    #     - PGDATA=/var/lib/postgresql/data/pgdata
    # volumes:
    #     - ./pgdata:/var/lib/postgresql/data/pgdata

  webserver:
    image: puckel/docker-airflow:1.10.9
    restart: always
    depends_on:
      - postgres
      - redis
    environment:
      - LOAD_EX=n
      - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
      - EXECUTOR=Celery
      # - POSTGRES_USER=airflow
      # - POSTGRES_PASSWORD=airflow
      # - POSTGRES_DB=airflow
      # - REDIS_PASSWORD=redispass
    volumes:
      - ./dags:/usr/local/airflow/dags
      # Uncomment to include custom plugins
      # - ./plugins:/usr/local/airflow/plugins
    ports:
      - "8080:8080"
    command: webserver
    healthcheck:
      test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow-webserver.pid ]"]
      interval: 30s
      timeout: 30s
      retries: 3

  flower:
    image: puckel/docker-airflow:1.10.9
    restart: always
    depends_on:
      - redis
    environment:
      - EXECUTOR=Celery
      # - REDIS_PASSWORD=redispass
    ports:
      - "5555:5555"
    command: flower

  scheduler:
    image: puckel/docker-airflow:1.10.9
    restart: always
    depends_on:
      - webserver
    volumes:
      - ./dags:/usr/local/airflow/dags
      # Uncomment to include custom plugins
      # - ./plugins:/usr/local/airflow/plugins
    environment:
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
      - LOAD_EX=n
      - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
      - EXECUTOR=Celery
      # - POSTGRES_USER=airflow
      # - POSTGRES_PASSWORD=airflow
      # - POSTGRES_DB=airflow
      # - REDIS_PASSWORD=redispass
    command: scheduler

  worker:
    image: puckel/docker-airflow:1.10.9
    restart: always
    depends_on:
      - scheduler
    volumes:
      - ./dags:/usr/local/airflow/dags
      # Uncomment to include custom plugins
      # - ./plugins:/usr/local/airflow/plugins
    environment:
      - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
      - EXECUTOR=Celery
      # - POSTGRES_USER=airflow
      # - POSTGRES_PASSWORD=airflow
      # - POSTGRES_DB=airflow
      # - REDIS_PASSWORD=redispass
    command: worker
```

Código-fonte 2 - Código do arquivo de configuração do Airflow
Fonte: Elaborado pelo autor

Execute um Docker ps para listar os containers em execução e pegar o ID do container do Airflow.

```
docker ps
```

Código-fonte 3 - Docker ps lista os contêineres em execução
Fonte: Elaborado pelo autor

O ID do container são os 3 primeiros dígitos da hash apresentada na coluna ID.

Vamos desenvolver nossas DAGs em python, que deve conter as tasks necessárias para atender nossa ingestão de dados e, quando finalizadas, enviamos as DAGs para o Airflow com os comandos abaixo:

```
docker cp dag_ingestao.py SEUID_AIRFLOW:usr/local/airflow/dags
```

Código-fonte 4 - Docker cp para copiar o arquivo da dag para o container do airflow
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O comando acima copia o arquivo dag_ingestao.py para o container do Airflow e salva no diretório usr/local/Airflow/dags.

Agora vamos escrever nossas dags.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

# 6 INGESTÃO DE DADOS

No SQL Server, trabalhamos com um cenário de Schemas. Um schema é para organizar os dados por assuntos. Se observarem, notarão que temos os schemas dbo (padrão SQL), HumanResources, Person, Production, Purchasing e Sales.

No nosso caso de estudo, trataremos cada um desses schemas como um banco de dados único. Imaginem que esses schemas (assuntos) estão em bancos de dados separados e unificaremos todos em um único local, como um data lake.

Abaixo, temos o diagrama das bases de dados que vamos movimentar.

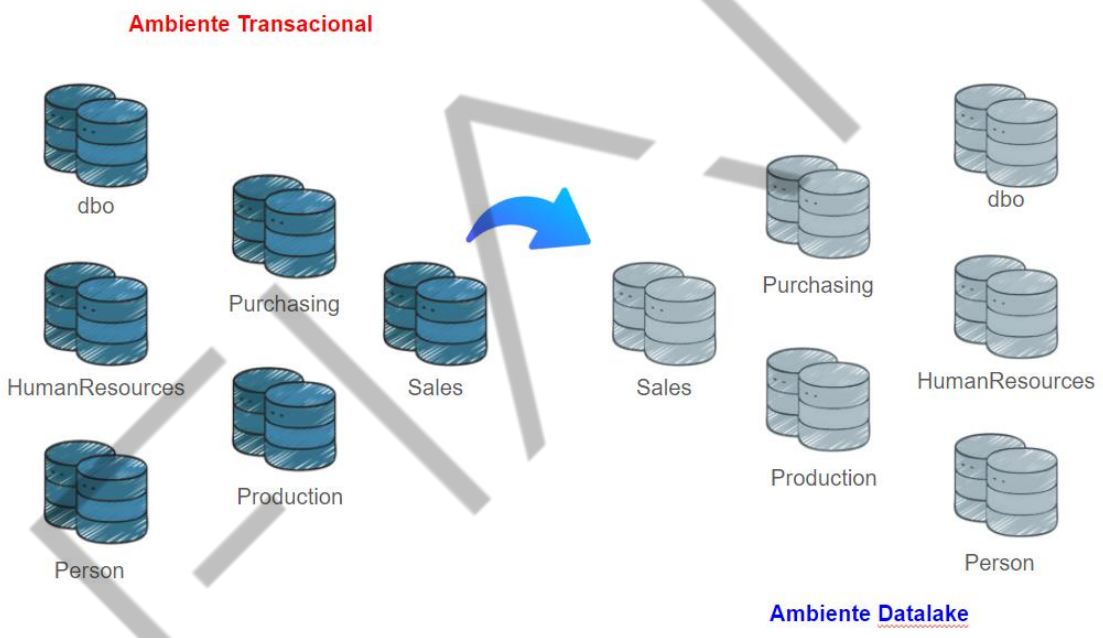


Figura 7 - Ingestão de dados do ambiente transacional para o datalake
Fonte: Elaborado pelo autor

Com esse cenário em mãos, criamos a DAG que movimenta os dados de forma FULL Load dos dados diariamente, ou seja, todos os dias os dados das tabelas são truncados e carregados novamente.

A primeira parte do código da nossa DAG é a importação das bibliotecas e definição de parâmetros da DAG. Notem que temos data de início, se temos dependências, quantidade de tentativas e de quanto em quanto tempo deve tentar novamente em caso de falha.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
import pyodbc
import pandas as pd

# Parâmetros do DAG
default_args = {
    'owner': 'airflow',
    'start_date': datetime(2023, 1, 1),
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

def truncate_table():
    # String de conexão, usa o driver ODBC para conectar no SQL Server
    string_conexao = f"DRIVER={{ODBC Driver 17 for SQL
Server}};SERVER={instsqlfiap1};DATABASE={AdventureWorks2022};Trusted_Connection=yes"
    connection = pyodbc.connect(connection_str)
    cursor = connection.cursor()
```

```
# Levantamos as tabelas de todos os schemas do banco de dados
    cursor.execute("SELECT table_name FROM information_schema.tables WHERE table_type =
'BASE TABLE'")
    tables = [row.table_name for row in cursor.fetchall()]

    # Fazemos o truncate em todas as tabelas
    for table in tables:
        truncate_query = f"TRUNCATE TABLE {table}"
        cursor.execute(truncate_query)

    # Fazemos commit e encerramos o cursor
    connection.commit()
    connection.close()

def load_data():
    # Conectar no banco de dados de origem
    connection_str1 = f"DRIVER={{ODBC Driver 17 for SQL
Server}};SERVER={instsqlfiap1};DATABASE={AdventureWorks2022};Trusted_Connection=yes"
    connection1 = pyodbc.connect(connection_str1)
    cursor1 = connection1.cursor()

    # Conectar no banco de dados destino
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
    connection_str2 = f"DRIVER={{ODBC Driver 17 for SQL
Server}};SERVER={instsqlfiap2};DATABASE={AdventureWorks2022};Trusted_Connection=yes"
    connection2 = pyodbc.connect(connection_str2)
    cursor2 = connection2.cursor()

    # Consultar todas as tabelas do primeiro banco de dados
    cursor1.execute("SELECT table_name FROM information_schema.tables WHERE table_type
= 'BASE TABLE'")
    tables = [row.table_name for row in cursor1.fetchall()]

    # Copiar dados de uma instância para outra
    for table in tables:
        select_query = f"SELECT * FROM {table}"
        data = cursor1.execute(select_query).fetchall()

        # Limpar a tabela de destino antes de carregar os dados
        cursor2.execute(f"TRUNCATE TABLE {table}")

        # Inserir dados na tabela de destino
        for row in data:
            cursor2.execute(f"INSERT INTO {table} VALUES (?, ?, ..., ?)", row)

    connection2.commit()
    connection1.close()
    connection2.close()

    # Truncate em todas as tabelas do banco de dados adventureworks na instância instsqlfiap2
    truncate_tables_in_instance()

    # Ler as tabelas de todos os schemas do banco de dados na instância instsqlfiap1 e carregar
nas tabelas da instância instsqlfiap2
    load_data_from_instance1_to_instance2()


# Definição do DAG
dag = DAG(
    'sql_server_full_load',
    default_args=default_args,
    description='DAG para carregar dados de um banco SQL Server para outro',
    schedule_interval=timedelta(days=1),  # Agendamento diário, ajuste conforme necessário
)

# Operador para execução da carga full
load_full_data_task = PythonOperator(
    task_id='load_full_data',
    python_callable=load_full_data,
    provide_context=True,
    dag=dag,
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
)

# Definir dependências se houver, como só temos uma task não temos depedencias
load_full_data_task

if __name__ == "__main__":
    dag.cli()
```

Código-fonte 5 - Código de criação da DAG
Fonte: Elaborado pelo autor

Def Truncate table é uma função que abre conexão com o banco de dados destino, lista as tabelas de todos os schemas e realiza o truncate table, ou seja, apaga os registros, mas mantém a estrutura da tabela.

Def load data é outra função que conecta no banco de origem e destino, aplica o truncate tabela nas tabelas por meio da função anterior e carrega os dados vindos do banco de dados de origem.

A parte final da DAG é a criação do operador e definição de dependências. Nesse caso, não temos operadores dependentes e nem sequência de execução dos operadores, mas poderíamos quebrar em truncate e carga em operadores separados e aí sim teríamos dependência de execução entre eles, e isso seria refletido no final do script da DAG.

Com a DAG pronta, basta copiar ela por meio do comando Docker cp dag_sdql_ingest.py ID_AIRFLOW:usr/local/Airflow/dags e acessar o endereço localhost:8080/admin que a DAG estará disponível e, nessa tela, poderemos ver suas execuções, com falhas e êxitos, analisar logs, ver o script que está sendo executado e todas as tarefas de administração desse flow.

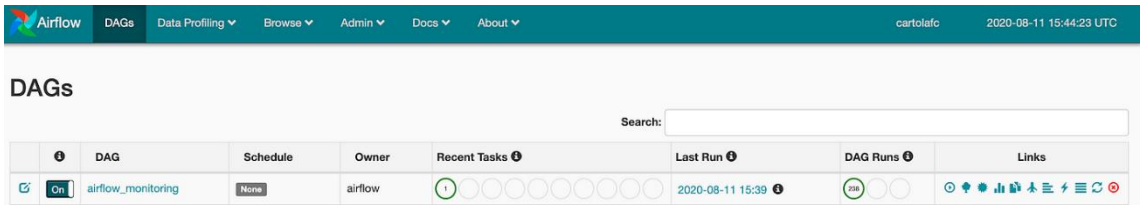


Figura 8 - Tela de gerenciamento do Airflow
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Neste capitulo, compreendemos melhor o que é uma ingestão de dados, etapas que fazem parte desse processo, algumas dicas importantes antes de começar e por onde começar. Entender o que será feito com o dado, o que será o resultado final dessa ingestão, nos ajudará a ter uma visão mais ampla do que precisa ser feito e de como fazer para atingir esse objetivo.

Criamos nossa primeira ingestão de dados utilizando bancos de dados SQL Server com uma carga Full Load de todas as tabelas de forma dinâmica, fazendo uma movimentação de dados FULL Load, onde os dados são truncados no destino, e carregamos novamente da origem para o destino com tudo monitorado e gerenciado pelo Airflow sendo executado em um Docker.

Esse cenário é bastante aplicado atualmente e pode nos gerar uma economia utilizando apenas o processamento de quando for executar nossos pipelines utilizando containers, e tudo isso ligado à uma esteira de integração contínua e desenvolvimento contínuo.

Essa atividade prática foi realizada utilizando Windows 11, Docker Desktop e SQL Server Developer. Caso esteja utilizando alguma distribuição Linux, a única coisa que mudaria seria a instalação do SQL Server, que deve ser diferente, mas o restante seguirá o mesmo passo a passo.

FIAP

**PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875**
**marcosdaguealmeida@outlook.com**

## REFERÊNCIAS

DATA Management Association International. **DMBOK**, 2023. Disponível em: <https://technicspub.com/dmbok/>. Acesso em: 03 dez. 2023.

MACEY, Tobias. **97 Things every Data Engineering Shoould Know**, 2021. Disponível em: <https://learning.oreilly.com/library/view/97-things-every/9781492062400/ch03.html>. Acesso em: 03 dez. 2023.

NEEDHAM, Mark. **Building Real-Time Analytics Systems**, 2023. Disponível em: <https://learning.oreilly.com/library/view/building-real-time-analytics/9781098138783/>. Acesso em: 04 dez. 2023.

REIS, Joe; HOUSLEY, Matt. **Fundamentals of Data Engineering**. 2022. Disponível em: <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/ch01.html#the_background_and_skills_of_a_data_eng/>. Acesso em: 04 dez. 2023.

FIAP

**PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875**
**marcosdaguealmeida@outlook.com**